

Introduzione e contesto dell'applicazione (0.5 pagine) 1 figura use case UML

Analisi dei requisiti (0.5 pagine) -> es. sicurezza ssl, keyclock, upload file

Back-end (1 pagina min, 2 pagine max): descr. tecnologie-> max 2 img flow diagram per parte transazionale

Front-end (1 pagina min, 2 pagine max): descr. tecnologie -> 3 screenshot max

Conclusioni (0.5 pagine max)

ESEMPIO (senza figure): Analisi e descrizione dell'applicazione “SpringProjectPSW”

1. Introduzione e contesto dell'applicazione

L'applicazione “SpringProjectPSW” rappresenta un esempio significativo di sistema web sviluppato mediante il framework Spring Boot. Il progetto si configura come una piattaforma gestionale riconducibile a un contesto di e-commerce simulato (Fake Store), con l'obiettivo di consentire la gestione di prodotti, utenti e transazioni di acquisto.

Il contesto in cui si inserisce l'applicazione è prevalentemente didattico, in quanto il progetto consente di esplorare in maniera concreta le principali tecnologie e metodologie di sviluppo moderne, tra cui l'architettura a livelli, la gestione della persistenza tramite ORM e l'integrazione tra backend e interfaccia utente. Tuttavia, le scelte progettuali adottate rendono il sistema potenzialmente estendibile anche a scenari reali, in cui sia necessario gestire flussi di dati complessi e interazioni utente.

Dal punto di vista funzionale, l'applicazione permette di gestire l'intero ciclo di vita delle entità principali del dominio, ovvero prodotti, utenti e acquisti. L'integrazione di servizi REST e di un'interfaccia utente realizzata con tecnologie server-side consente inoltre di supportare sia interazioni dirette da parte degli utenti sia comunicazioni tra sistemi.

2. Analisi dei requisiti

L'analisi dei requisiti dell'applicazione può essere condotta osservando le funzionalità implementate e la struttura del codice. I requisiti funzionali principali riguardano la gestione dei prodotti, degli utenti e delle operazioni di acquisto. In particolare, il sistema consente l'inserimento e la consultazione dei prodotti, la registrazione degli utenti e la creazione di transazioni che associano uno o più prodotti a un acquisto.

Un aspetto rilevante è la gestione dell'unicità di alcuni attributi, come l'indirizzo email degli utenti e il codice a barre dei prodotti, che viene garantita attraverso controlli applicativi e mediante l'utilizzo di eccezioni specifiche. Questo evidenzia l'attenzione posta alla consistenza dei dati e alla prevenzione di errori logici.

Dal punto di vista delle operazioni, il sistema supporta le classiche funzionalità CRUD (Create, Read, Update, Delete), esposte tramite API REST e utilizzate anche dall'interfaccia utente. La presenza di controller dedicati alla gestione delle richieste HTTP dimostra una chiara separazione tra il livello di presentazione e la logica applicativa.

Accanto ai requisiti funzionali, emergono anche importanti requisiti non funzionali. Tra questi, la modularità del sistema è garantita dalla suddivisione in livelli, che facilita la manutenzione e l'evoluzione del codice. La scalabilità è supportata dalla struttura del progetto e dall'utilizzo di tecnologie come Spring Boot e Spring Data JPA, che permettono di gestire applicazioni anche di dimensioni maggiori. La robustezza è assicurata

dalla gestione strutturata delle eccezioni, mentre la sicurezza è affrontata tramite l'integrazione di Spring Security.

Particolarmente rilevante è proprio la gestione degli errori, implementata attraverso eccezioni personalizzate che permettono di distinguere tra diverse tipologie di problemi, come l'inserimento di dati duplicati o la richiesta di risorse inesistenti. Questo approccio contribuisce a migliorare la qualità complessiva del sistema e a rendere più chiaro il flusso.

3. Back-end e tecnologie impiegate

Il backend dell'applicazione costituisce il nucleo principale del sistema ed è realizzato secondo una classica architettura a livelli. Questa scelta progettuale consente di separare le responsabilità tra i diversi componenti, migliorando la leggibilità del codice e facilitando eventuali modifiche future.

Il livello dei controller rappresenta il punto di ingresso dell'applicazione e si occupa di gestire le richieste HTTP provenienti dal client. In questo progetto sono presenti sia controller REST, che espongono API utilizzabili da client esterni, sia componenti dedicati all'interfaccia utente sviluppata con Vaadin. I controller delegano la logica applicativa ai servizi, mantenendo così una chiara distinzione tra gestione delle richieste e elaborazione dei dati.

Il service layer svolge un ruolo centrale, in quanto contiene la logica di business dell'applicazione. In questo livello vengono implementate le operazioni principali, come la gestione dei prodotti, degli utenti e degli acquisti. I servizi coordinano l'interazione tra i controller e i repository, garantendo che le operazioni siano eseguite nel rispetto delle regole applicative.

Il livello di accesso ai dati è gestito tramite repository basati su Spring Data JPA. Questa tecnologia consente di interagire con il database in modo dichiarativo, riducendo la necessità di scrivere codice SQL esplicito. I repository permettono di eseguire operazioni di persistenza in maniera semplice ed efficiente, sfruttando le funzionalità offerte dal framework.

Il modello dati è rappresentato da un insieme di entità che riflettono il dominio applicativo. Le entità principali includono prodotti, utenti e acquisti, oltre a una classe intermedia che gestisce la relazione tra prodotti e acquisti. Questa soluzione consente di modellare correttamente relazioni complesse, come quelle N:N, mantenendo la coerenza del DB.

Dal punto di vista tecnologico, il backend si basa su Spring Boot, che semplifica la configurazione e l'avvio dell'applicazione grazie all'uso di convenzioni e configurazioni automatiche. L'integrazione con Spring MVC permette di gestire le richieste web, mentre Spring Data JPA facilita la persistenza dei dati. L'utilizzo di Spring Security consente di introdurre meccanismi di autenticazione e autorizzazione, aumentando il livello di protezione.

La configurazione dell'applicazione è gestita tramite file dedicati e classi di configurazione, che permettono di definire parametri come la connessione al database e le regole di sicurezza. Questo approccio favorisce la separazione tra configurazione e logica applicativa, rendendo il sistema più flessibile.

4. Front-end e tecnologie impiegate

Il frontend dell'applicazione è realizzato utilizzando Vaadin, un framework che consente di sviluppare interfacce utente web direttamente in Java. Questa scelta rappresenta un'alternativa ai tradizionali

framework basati su JavaScript, permettendo agli sviluppatori di gestire l'interfaccia grafica utilizzando lo stesso linguaggio del backend.

L'architettura adottata è di tipo server-side, in cui la logica dell'interfaccia viene eseguita sul server e il client si occupa principalmente della visualizzazione. Questo approccio semplifica lo sviluppo e riduce la complessità legata alla gestione dello stato dell'applicazione client.

Le principali componenti del frontend includono la classe principale dell'applicazione e moduli dedicati alla ricerca dei prodotti e alla registrazione degli utenti. Attraverso queste componenti, l'utente può interagire con il sistema in modo intuitivo, eseguendo operazioni come la consultazione dei prodotti e l'inserimento dei propri dati.

La presenza di file legati a Node.js suggerisce l'utilizzo di strumenti di supporto per la gestione delle dipendenze e del processo di build del frontend, anche se la logica principale rimane gestita lato server. Questo consente di integrare eventuali funzionalità avanzate senza introdurre un'eccessiva complessità.

Dal punto di vista dell'esperienza utente, l'interfaccia risulta semplice ma funzionale, coerente con gli obiettivi didattici del progetto. L'integrazione diretta con il backend permette una comunicazione immediata tra interfaccia e logica applicativa, riducendo la necessità di livelli intermedi.

5. Considerazioni finali

L'applicazione "SpringProjectPSW" costituisce un esempio completo e ben strutturato di sistema web sviluppato con Spring Boot. L'adozione di un'architettura a livelli, l'utilizzo di tecnologie consolidate e la gestione accurata delle entità e delle relazioni contribuiscono a rendere il progetto solido e facilmente estendibile.

Uno dei principali punti di forza dell'applicazione è rappresentato dalla chiara separazione delle responsabilità, che consente di mantenere il codice organizzato e di facilitare eventuali modifiche. L'integrazione tra backend e frontend tramite Vaadin rappresenta inoltre una scelta interessante, in quanto semplifica lo sviluppo dell'interfaccia utente.

Pur trattandosi di un progetto con finalità didattiche, l'applicazione presenta caratteristiche che la rendono adatta a scenari reali. Con opportuni miglioramenti, come l'introduzione di test automatizzati, il potenziamento della sicurezza e l'ottimizzazione dell'interfaccia utente, il sistema potrebbe essere utilizzato come base per applicazioni più complesse.

In conclusione, il progetto analizzato dimostra come sia possibile sviluppare applicazioni web moderne e modulari utilizzando il framework Spring Boot, evidenziando al contempo l'importanza di una progettazione strutturata e dell'utilizzo di strumenti adeguati per garantire qualità, manutenibilità e scalabilità del software.